



SDA WEEK 10

ARCHITECTURAL MODELING: 4+1 PERSPECTIVE APPROACH

Syed Ahmed Khan
syed.ahmed@nu.edu.pk

LEARNING OUTCOMES

1. Explain why software architecture needs multiple views
2. Describe the 4+1 View Model by Philippe Kruchten
3. Differentiate between:
 - a. Logical View
 - b. Process View
 - c. Development View
 - d. Physical View
4. Scenarios / Use Cases
5. Map UML diagrams to each architectural view
6. Understand how the 4+1 model supports communication, design clarity, and documentation

WHY SO MANY UML DIAGRAMS?

UML packages/diagrams

- Use Case Diagram
- Class Diagram
- Activity Diagram
- Sequence Diagram
- Component Diagram
- Deployment Diagram

WHY SO MANY UML DIAGRAMS?

- Why can't one diagram explain the whole system?

“This is the exact question the 4+1 model helps answer.”

A SOFTWARE SYSTEM HAS MANY PERSPECTIVES

A software system can be viewed in terms of:

- What it does
- How it behaves
- How it is built
- Where it runs
- How users interact with it

WHAT IS AN ARCHITECTURAL VIEW?

Definition:

An architectural view is a representation of the system from the perspective of a particular concern or stakeholder.

WHAT IS AN ARCHITECTURAL VIEW?

Examples of stakeholders

- Users
- Developers
- Architects
- Testers
- DevOps / Deployment team

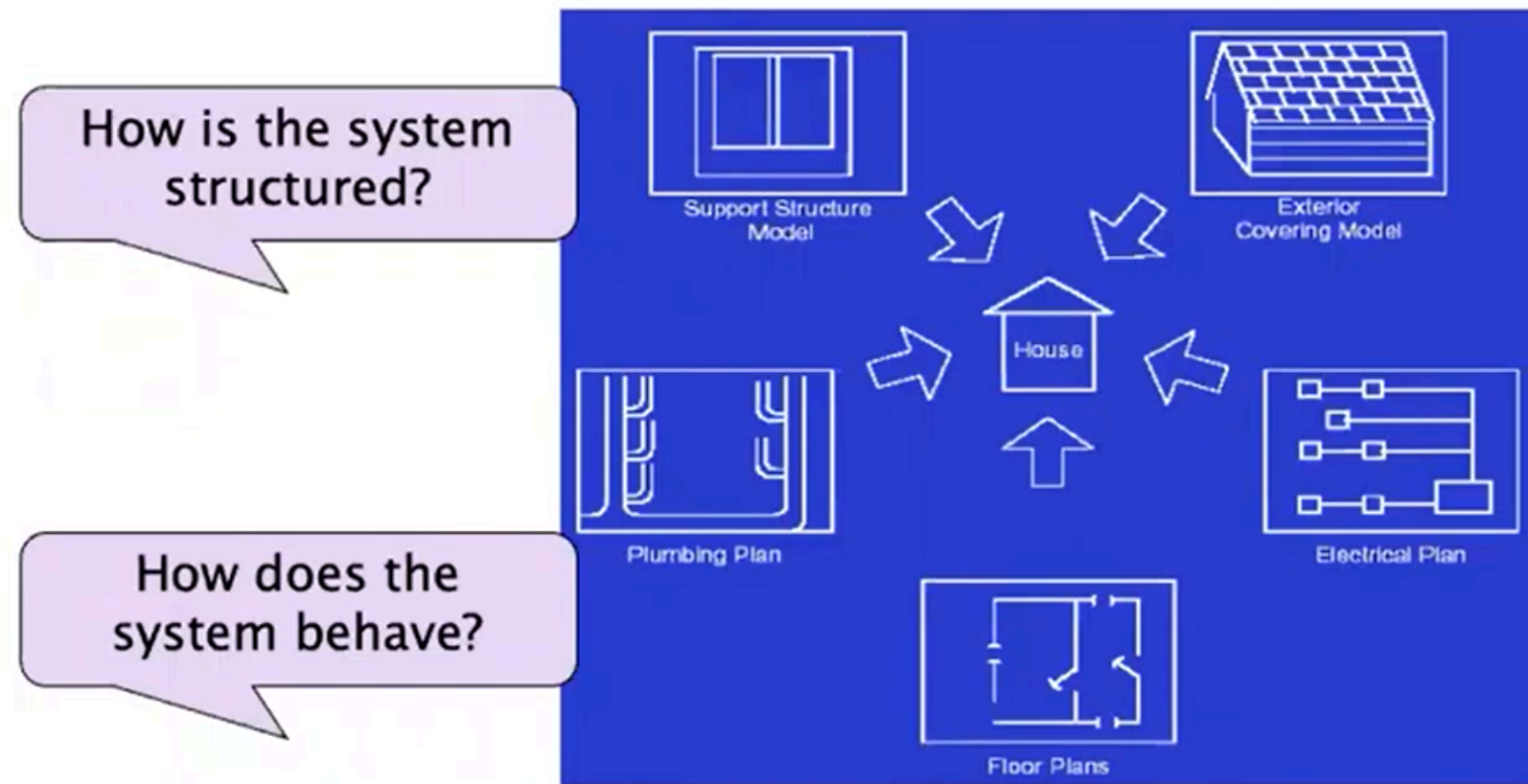


WHAT IS AN ARCHITECTURAL VIEW?

Different people care about different aspects of the same system.

TYPES OF VIEWS

- Structural view
- Dynamic view
- Physical view



A single language does not fit to capture all aspects of software systems

ARCHITECTURE IS NOT ONE DIAGRAM

A good software architecture should answer:

- What are the major system concepts?
- How do parts interact at runtime?
- How is the software organized for development?
- Where will it be deployed?
- Which scenarios must it support?

4+1 ARCHITECTURAL VIEW MODEL

Philippe Kruchten is a renowned Canadian software engineer and professor. He developed this model while working at Rational Software, where he contributed heavily to the Rational Unified Process (RUP).

His 4+1 model became a standard reference for describing software architectures clearly and systematically.

4+1 VIEW MODEL RATIONALE

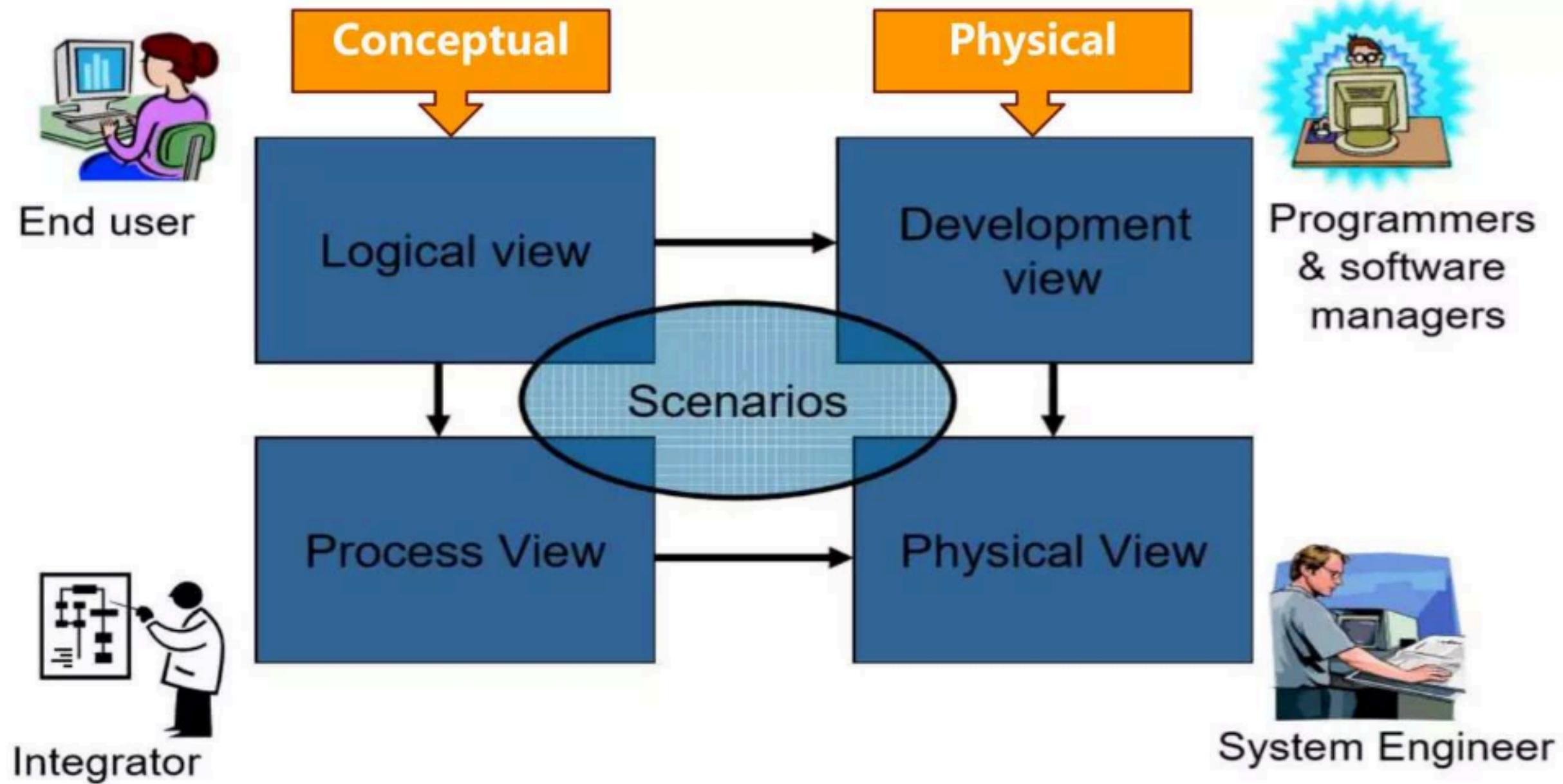
- A single, monolithic model is ineffective for communication.
- The 4+1 Architecture partitions the design into discrete, stakeholder-specific Views.
- When we build software systems, they're rarely simple. Stakeholders: users, developers, project managers, and system engineers, all view the system differently.
- These views run concurrently, meaning they are developed and refined in parallel.
- This separation of concerns is fundamental to managing large projects.

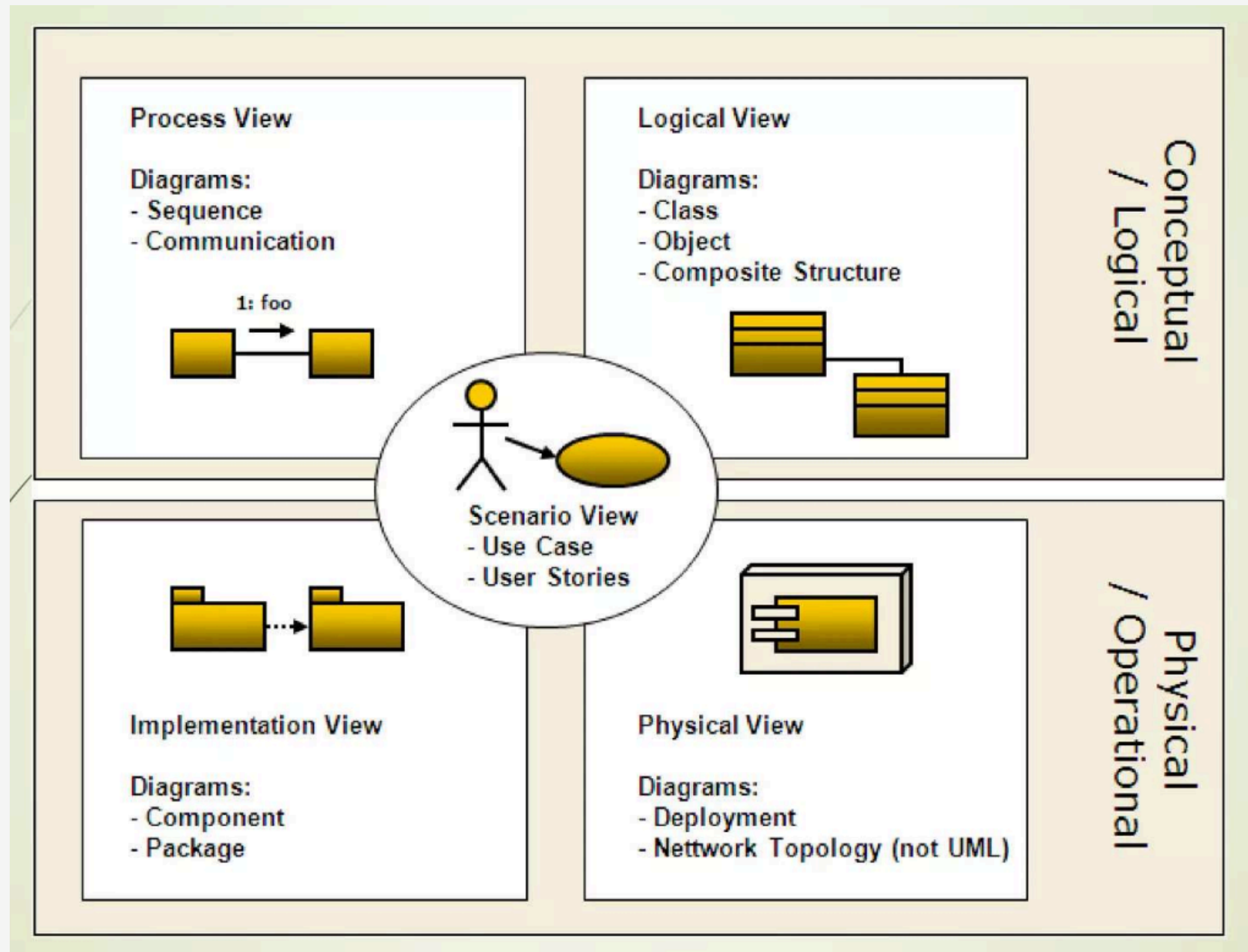
THE FIVE VIEWS

The model consists of four main views, plus one more that ties everything together – hence the name 4+1.

- Logical View - focuses on functionality for end users.
- Development View - focuses on implementation for developers.
- Process View - focuses on dynamic behavior and performance.
- Physical View - focuses on deployment and system topology.
- +1: Scenarios or Use Cases - tie all views together through examples of real system behavior.

4+1 View Model of Architecture





VIEW 1: LOGICAL VIEW

- It's based on object-oriented decomposition: identifying key abstractions, objects, and their interactions.
- Focuses on what are the major concepts and structures of the system?
- Typical UML diagrams used are class diagrams and Object diagrams.

VIEW 1: LOGICAL VIEW

Logical View Example

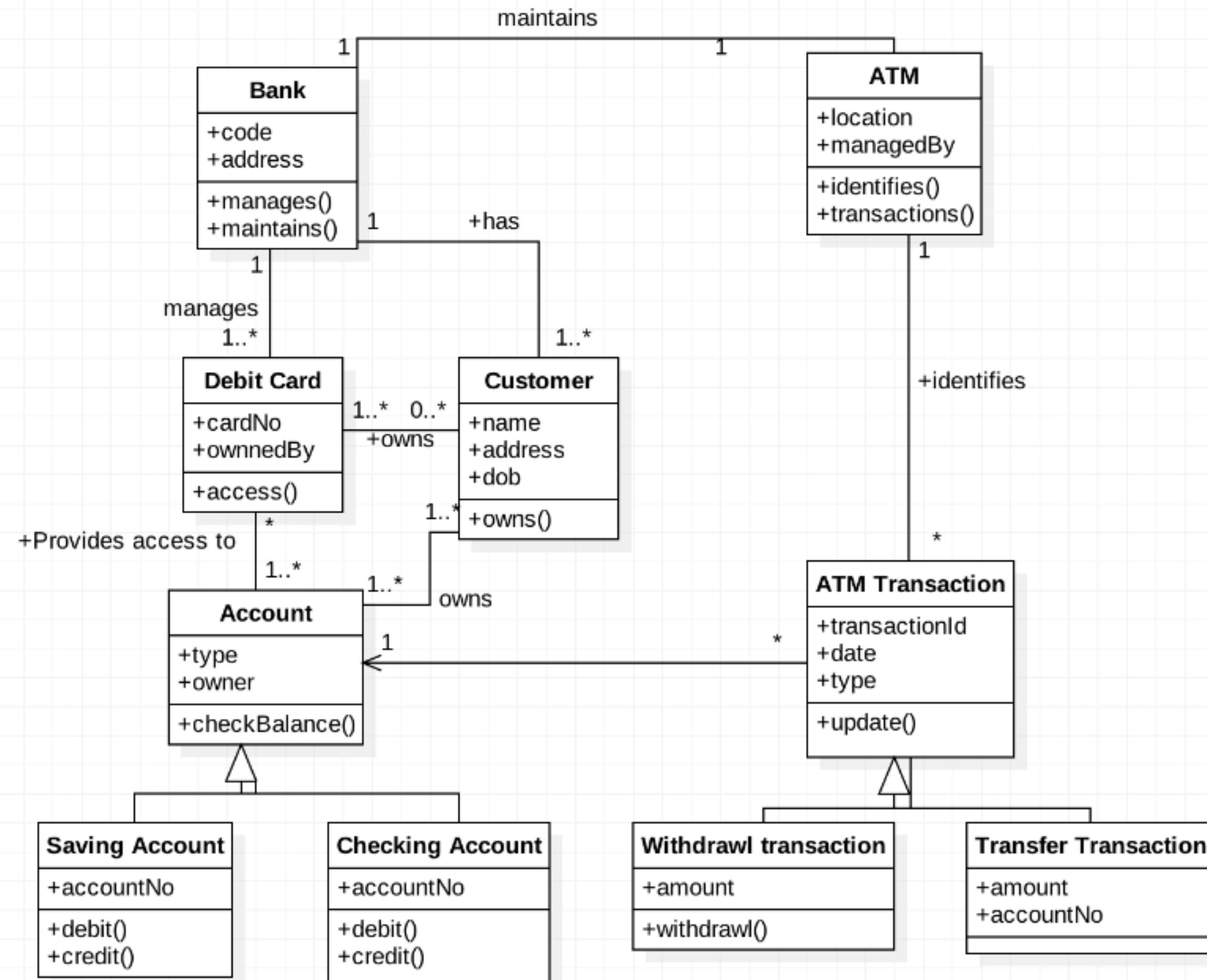
Scenario

Library Management System

Possible classes

- Book
- Member
- Librarian
- Loan
- Reservation
- Fine

CLASS DIAGRAM ATM



VIEW 2: PROCESS VIEW

- This view is all about How does the system behave while running?
- Questions this view answers:
 - How do parts communicate?
 - What happens concurrently?
 - How are events/messages handled?
 - How is synchronization achieved?
- Typical UML: Sequence Diagram, Activity Diagram, State Diagram and Communication Diagram

VIEW 2: PROCESS VIEW CONCERNS

- Concurrency
- Synchronization
- Parallel execution
- Signals / events
- Synchronous vs asynchronous communication
- Runtime interactions

VIEW 2: PROCESS VIEW

Process View Example

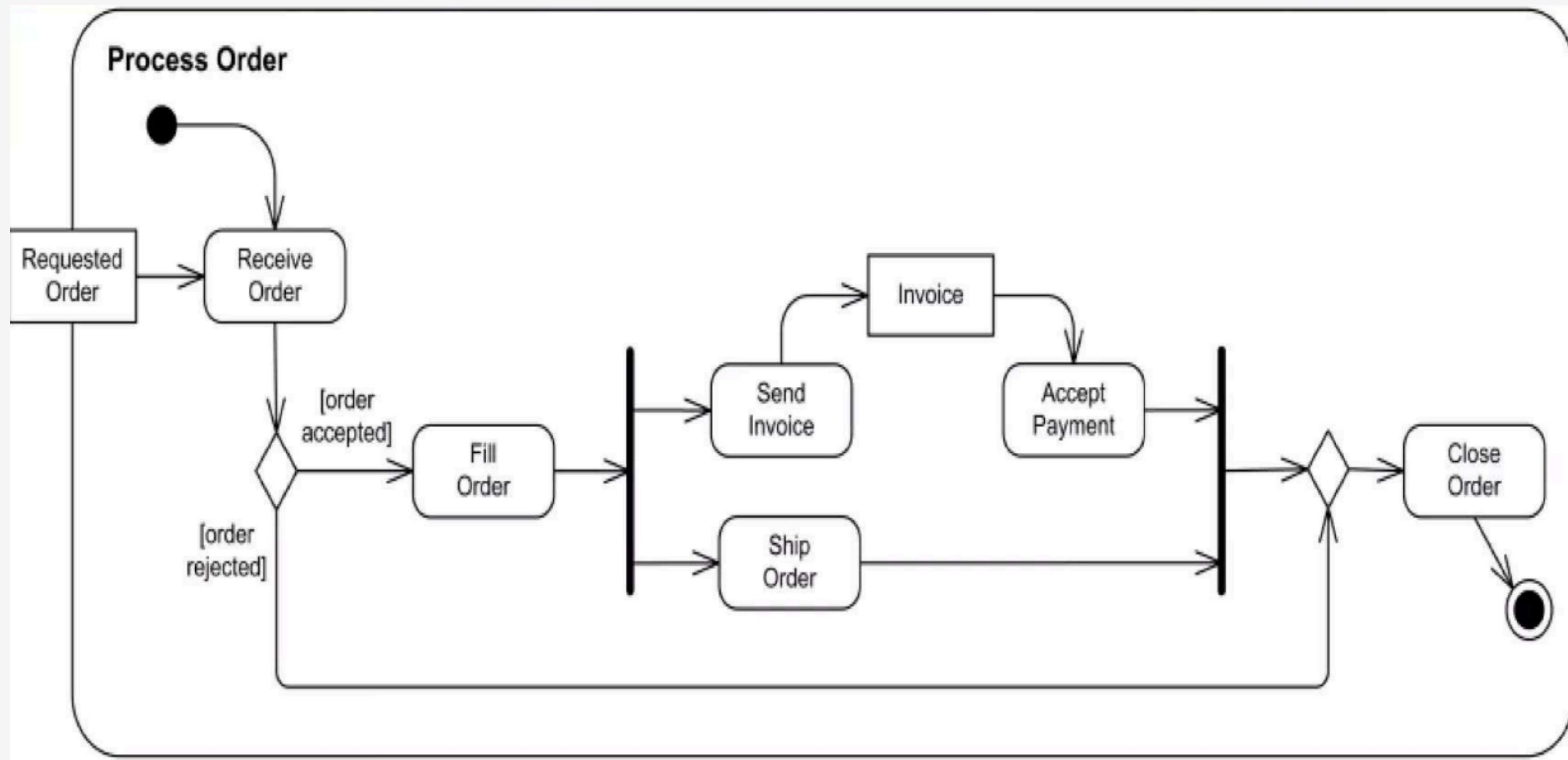
Scenario

ATM Withdrawal

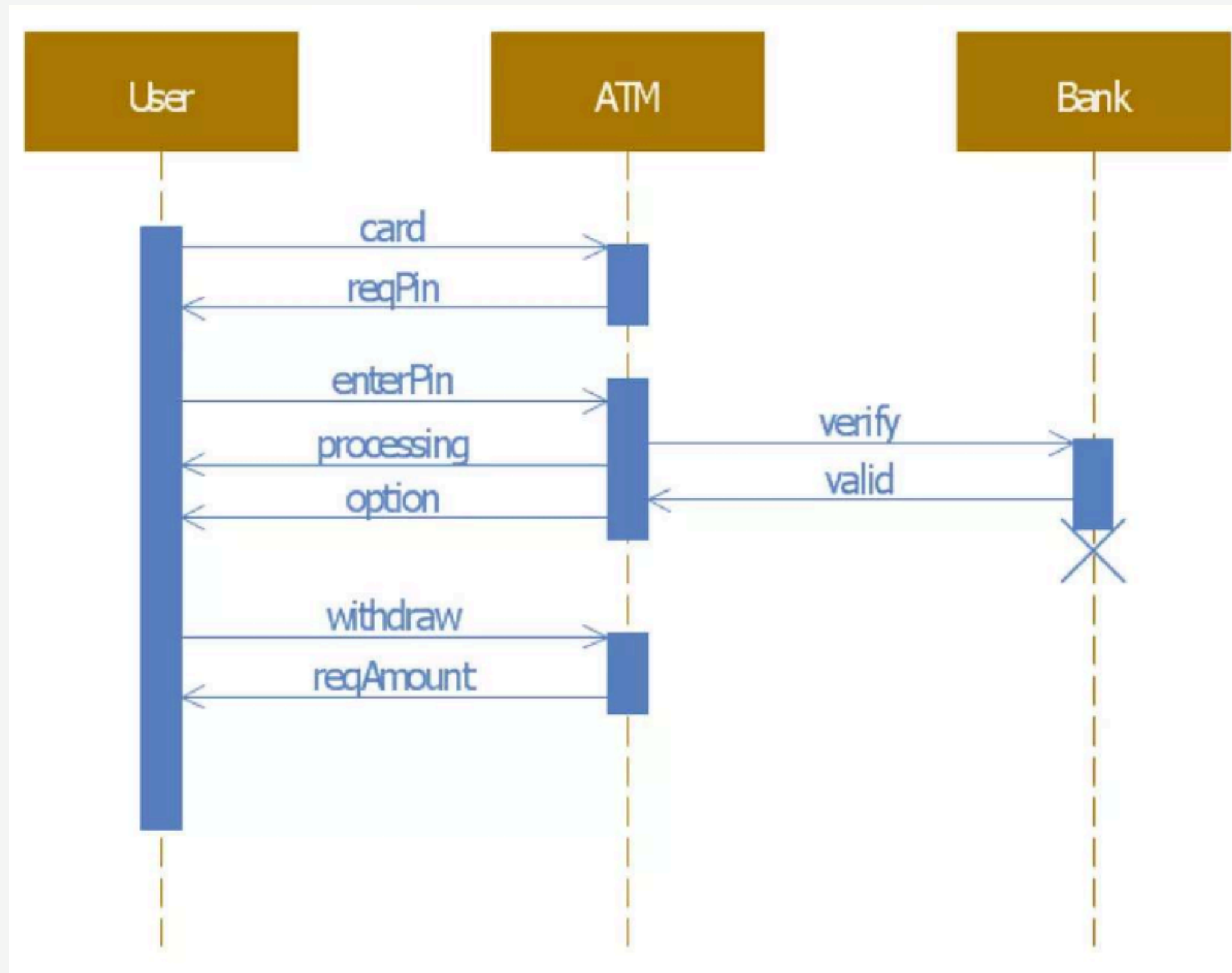
Runtime questions

- What happens after card insertion?
- How does ATM communicate with bank server?
- What if authorization is delayed?
- What happens on timeout?

PROCESS VIEW EXAMPLE



SEQUENCE DIAGRAM ATM



DEVELOPMENT VIEW

- The Development View, sometimes called the Implementation View, describes the software's static organization.
- It focuses on the actual code structure, modules, components, and layers.
- Programmers and software managers use this view to manage development, assign tasks, and plan releases.
- Key Tool: UML Component and package Diagrams

DEVELOPMENT VIEW

Development View Example

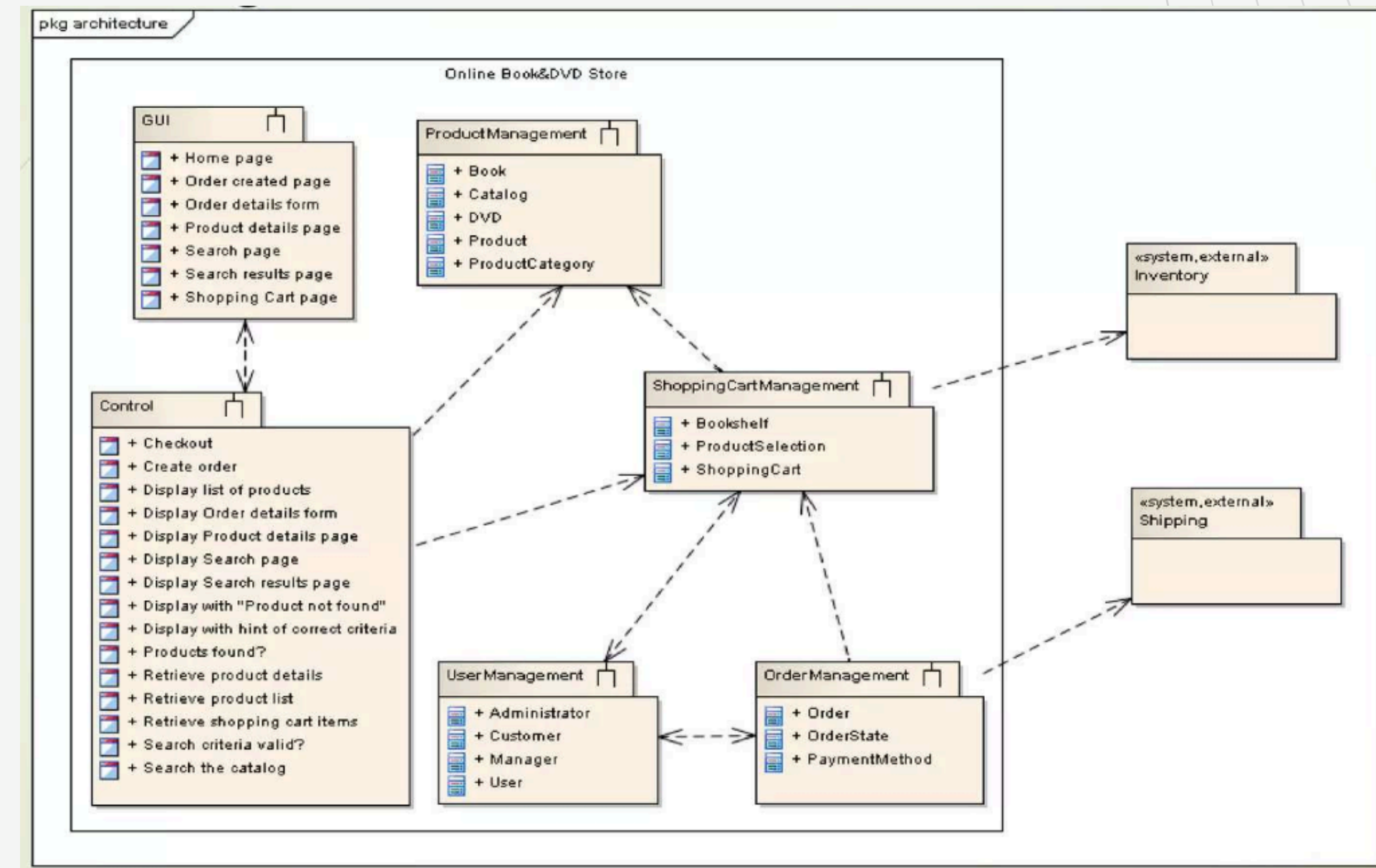
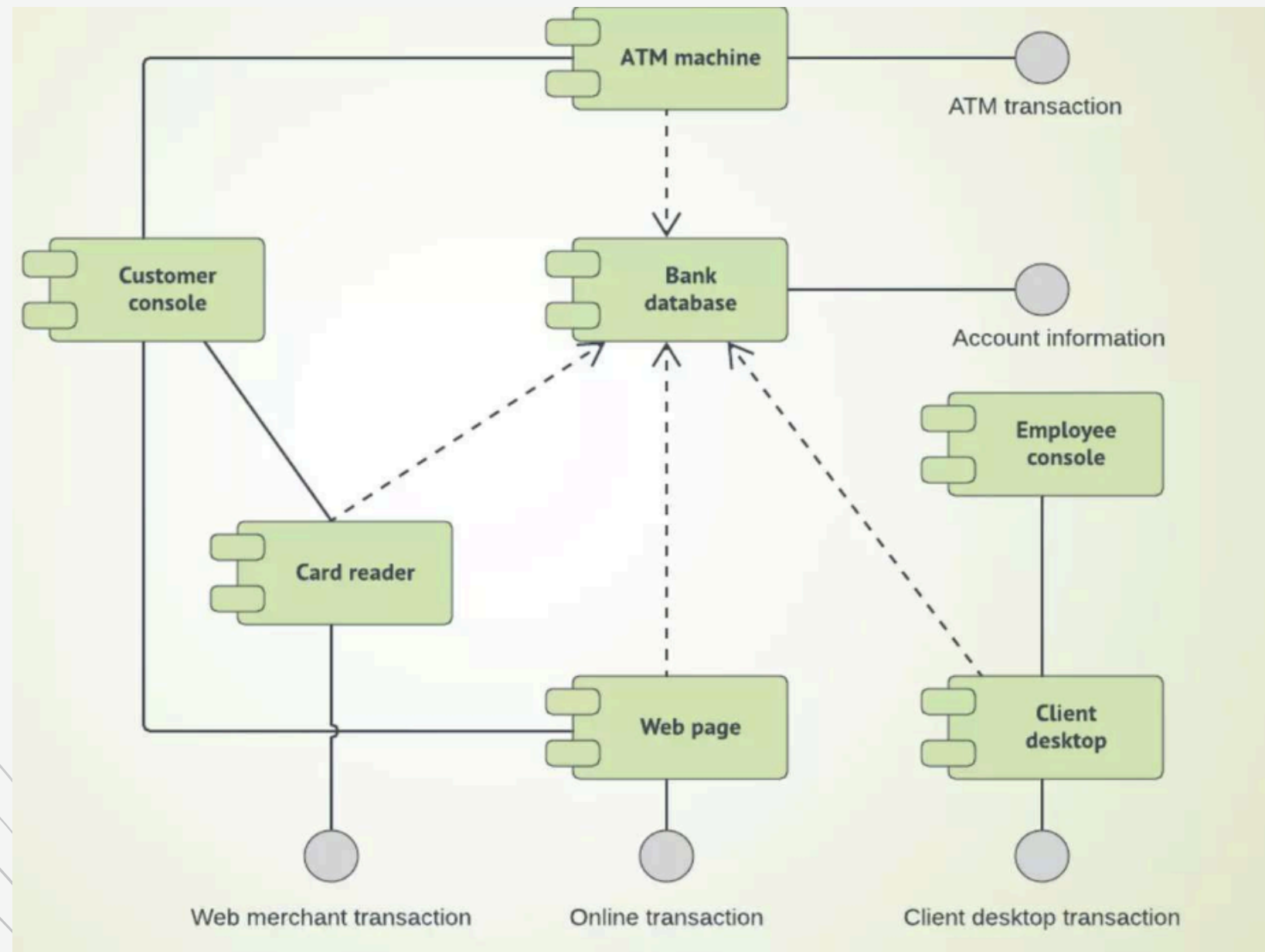
Scenario

Career Development Platform

Possible components

- User Management
- Resume Builder
- Job Matching
- Messaging
- Analytics

COMPONENT & PACKAGE DIAGRAMS



PHYSICAL VIEW

- The Physical View also called the Deployment View, describes how software components map onto hardware nodes.
- It's crucial for system engineers dealing with distributed systems.
- It defines the topology: which servers, networks, and devices host which components.
- It focuses on, where does the software run in the real world?

PHYSICAL VIEW

Physical View Example

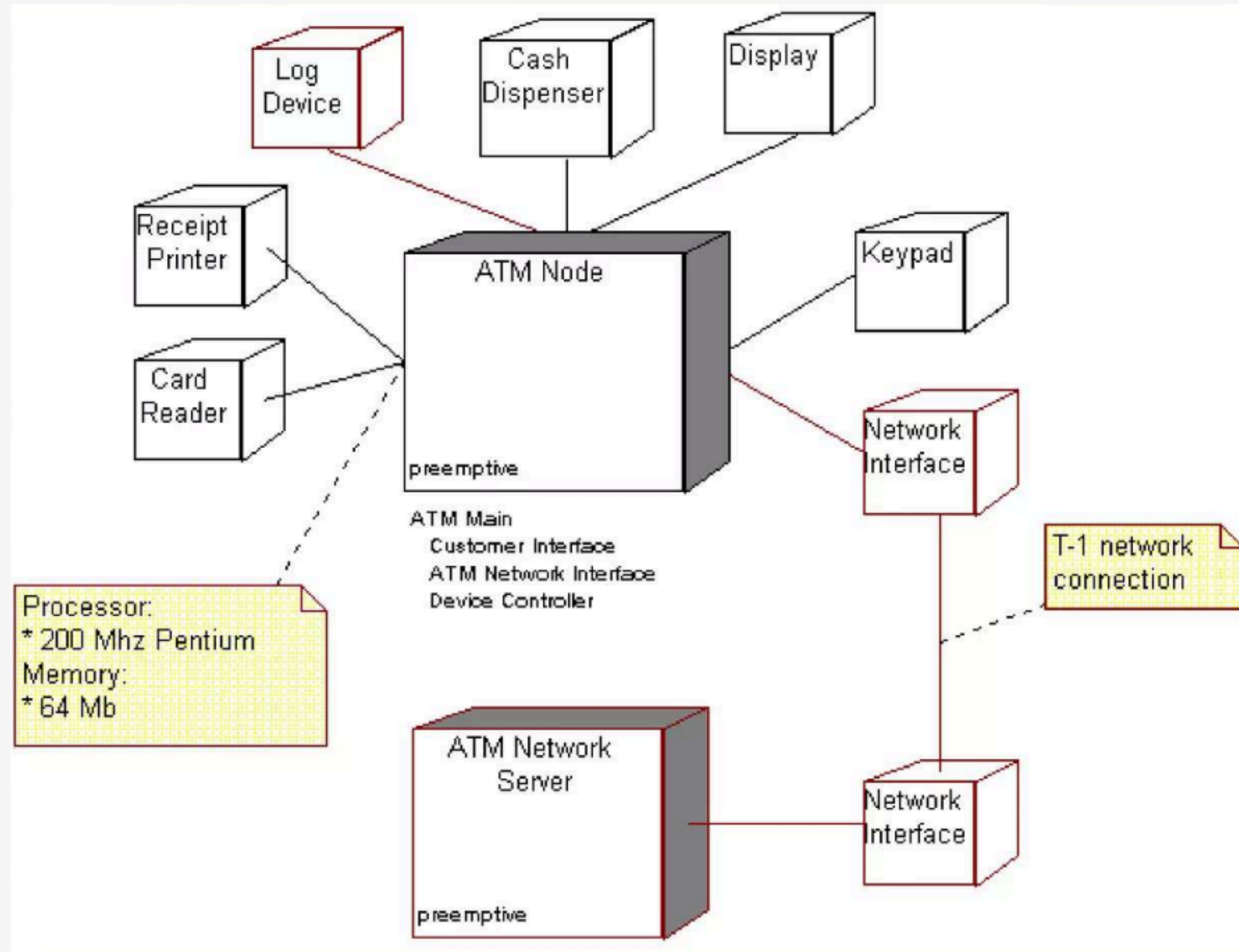
Scenario

Online Food Delivery System

Possible deployment nodes

- Customer Mobile App
- Restaurant Web Portal
- Backend Server
- Database Server
- Notification Service

PHYSICAL VIEW



THE +1: SCENARIOS / USE CASES

- Finally, we have the +1 View. The Scenario or Use Case View.
- This ties everything together by describing how the system behaves in concrete usage examples.
- Scenarios validate the architecture, they ensure consistency and completeness across all four views.
- Focuses on:
 - What important workflows must the system support?
 - What do users do?
 - What are the system's key functions?
 - Which interactions validate the architecture?

THE +1: SCENARIOS / USE CASES

Typical UML

- Use Case Diagram
- System Sequence Diagram
- Activity Diagram

This is the user-centered validation layer of the architecture.

WHY THE “+1” IS CRITICAL

Scenarios help validate whether the architecture supports:

- functional requirements
- major workflows
- system interactions
- user goals

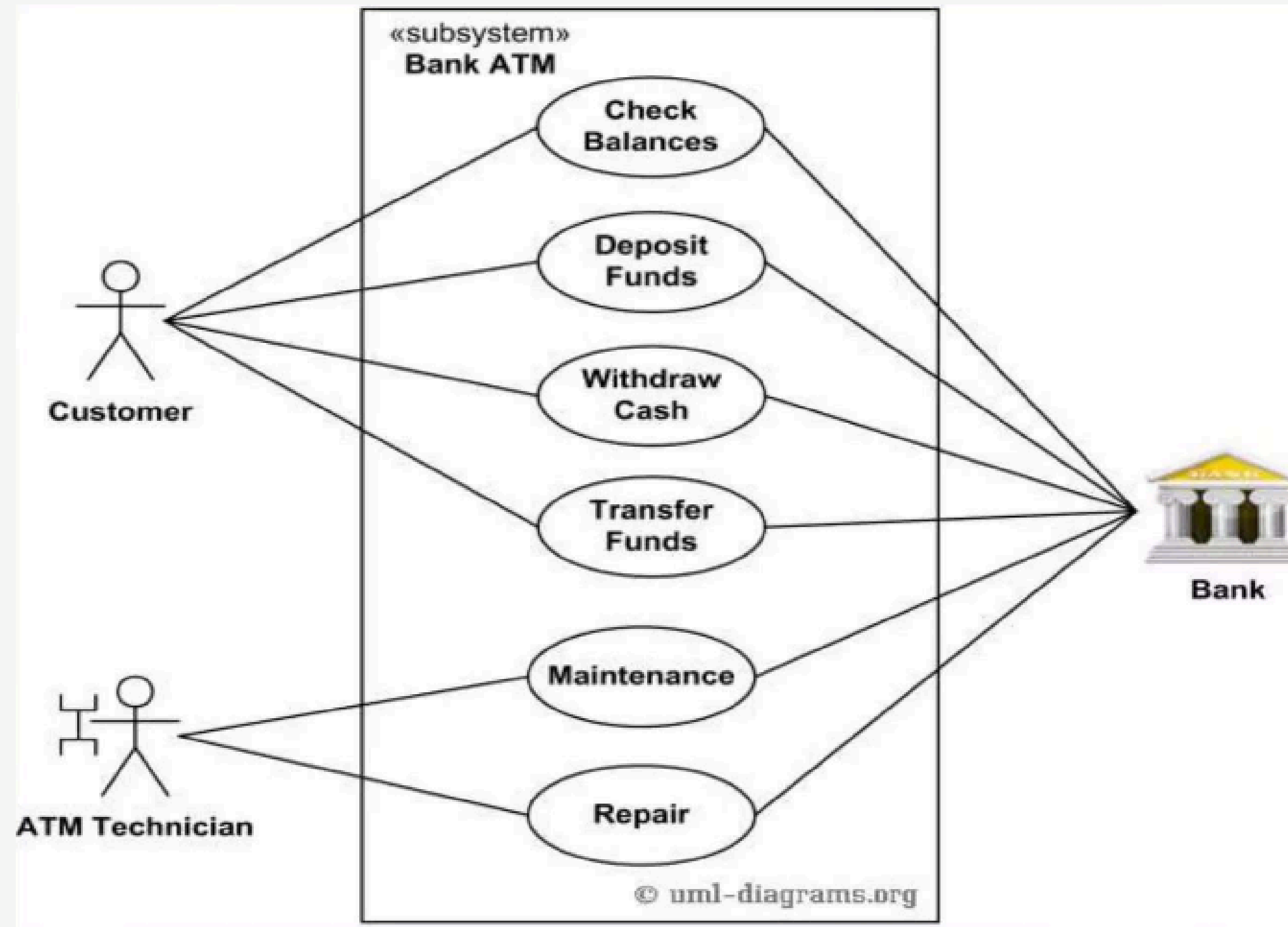
Architecture is useful only if it supports real scenarios.

Scenarios are not extra decoration – they are the reason architecture exists.

SCENARIO VIEW (4+1 VIEW)

- Finally, we have the +1 View. The Scenario or Use Case View.
- This ties everything together by describing how the system behaves in concrete usage examples.
- Scenarios validate the architecture, they ensure consistency and completeness across all four views.
- Use Case Example:
 - A customer withdrawing cash from an ATM.
 - This scenario involves logical classes, runtime processes, deployed components, and hardware nodes – integrating all perspectives.

SCENARIO VIEW (4+1 VIEW)





HOW THE VIEWS COMPLEMENT EACH OTHER

Each view answers a different but necessary architectural question.

View	Main Question
Logical	What exists in the system?
Process	How does it behave at runtime?
Development	How is the software organized?
Physical	Where does it run?
Scenarios	What must the system do?

HOW THE VIEWS COMPLEMENT EACH OTHER

Each view answers a different but necessary architectural question.

View	Common UML Diagrams
Logical View	Class Diagram, Object Diagram
Process View	Sequence, Activity, State, Communication
Development View	Component Diagram, Package Diagram
Physical View	Deployment Diagram
Scenarios / +1	Use Case Diagram, SSD, Activity Diagram

IMPORTANT NOTE

A Diagram Can Support More Than One View

Content

Examples:

- Activity Diagram → Process View / Scenario View
- Sequence Diagram → Process View / Logical support
- State Diagram → Process behavior / object lifecycle

SMART CAMPUS – LOGICAL VIEW

What are the main entities and relationships?

- Student
- Faculty
- Course
- Registration
- Payment
- Complaint
- AttendanceRecord

SMART CAMPUS – PROCESS VIEW

Possible runtime interactions

- Student registers for course
- Payment is processed
- Complaint is submitted and escalated
- Attendance is marked live

Possible diagrams

- Sequence Diagram
- Activity Diagram
- State Diagram

SMART CAMPUS – DEVELOPMENT VIEW

Possible components

- Authentication Component
- Course Management Component
- Payment Component
- Complaint Handling Component
- Attendance Component

SMART CAMPUS – PHYSICAL VIEW

Possible deployment nodes

- Student Mobile App
- Faculty Portal
- Backend Server
- Database Server
- Notification Server

SMART CAMPUS – SCENARIOS / +1

Possible use cases

- Register for Course
- Pay Semester Fee
- Submit Complaint
- Mark Attendance

These scenarios validate whether all other views are meaningful.

WHY THIS MATTERS IN REAL INDUSTRY

Different teams naturally think in different views:

- Product teams → features / use cases
- Developers → classes / components
- Backend teams → runtime behavior
- DevOps teams → deployment

CLOSING THOUGHTS

A software system is one reality, but many useful views.”

For your project, ask yourself:

- What is my system’s logical view?
- What is its process view?
- What is its development view?
- What is its physical view?
- What scenarios validate it?

CLASS ACTIVITY

Task: For your course project, fill the following:

View	What would you model?
Logical View	Main classes/entities
Process View	Main workflows/interactions
Development View	Main components/modules
Physical View	Main nodes/devices
Scenarios	3 key use cases



THANK YOU